

[Accenture]

# Guida Operativa Claude Registry

87 agenti · 42 skill · Workflow BMAD

Versione 3.0 · 31 maggio 2026

Classificazione: Interno — Solo uso team

Autore: document-creator agent

Revisione: claude-registry maintainers

# Controllo del Documento

| Versione | Data        | Autore           | Modifiche principali                                                                                                                                                         |
|----------|-------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0      | Apr 2026    | document-creator | Prima emissione — Parti 1 e 2 (utilizzo e contribuzione)                                                                                                                     |
| 2.0      | Mag 2026    | document-creator | 83 agenti, pipeline replatforming Fasi 0–4, skill layer, BMAD intro                                                                                                          |
| 3.0      | 31 mag 2026 | document-creator | 87 agenti, 42 skill; Parte 3 BMAD completa; Phase 5 rimossa (assorbita in Fase 4 Step 6); test-data-seeder (Step 5.5); progressive disclosure; document-as-cache; eval suite |

**Nota:** I conteggi di agenti e skill sono quelli correnti al momento della generazione. La tabella aggiornata si trova sempre in `claude-marketplace/catalog.json`.

## Sommario

### Parte 1 — Utilizzo delle capability

- 1. Prerequisiti e installazione
- 2. Trovare la capability giusta
- 3. Verifica e configurazione
- 4. Personalizzazione per progetto
- 5. Aggiornamenti

### Parte 2 — Contribuire al registry

- 6. Struttura del repository
- 7. Creare una nuova capability
- 8. Scrivere il system prompt
- 9. Test e validazione locale
- 10. PR, review e pubblicazione
- 11. Governance e versionamento

### Parte 3 — Metodologia BMAD

- 12. Cos'è il layer BMAD
- 13. Progressive disclosure
- 14. Document-as-cache

- 15. Workflow registry
- 16. DAG in catalog.json
- 17. Eval suite
- 18. Aggiungere un nuovo use case
- 19. Il workflow Application Replatforming

## Appendice

- A. Tabella completa agenti (87)
- B. Tabella skill (42)
- C. Comandi di riferimento rapido
- D. Convenzioni di naming e colori

PARTE 1

# Utilizzo delle Capability

---

Come trovare, installare, verificare e aggiornare gli agenti e le skill del registry nel proprio progetto.

# 1. Prerequisiti e installazione

## 1.1 Prerequisiti

- **Claude Code** installato e configurato (versione con supporto subagenti)
- Accesso in lettura al repository `claude-registry`
- `bash` disponibile nel terminale (macOS / Linux / WSL)

## 1.2 Installazione con setup script (consigliata)

Lo script interattivo è il modo più semplice per installare le capability. Risolve automaticamente tutte le dipendenze di skill.

```
./claude-catalog/scripts/setup-capabilities.sh /percorso/al/tuo-progetto
```

Lo script:

- Mostra la lista completa delle capability con tier ( `stable` / `beta` )
- Permette di selezionare per numero o di digitare `all` per installare tutte le capability stable
- Installa automaticamente le skill dipendenti (es. selezionando `developer-java`, installa anche `java-spring-standards`, `testing-standards`, `rest-api-standards` )
- Aggiorna `.claude/settings.json` del progetto con le regole di permesso `Agent (name)`

**Installazione globale:** Per rendere tutte le capability disponibili in qualsiasi progetto ( `~/claude/agents/` ), usa:

```
./claude-catalog/scripts/setup-capabilities.sh --global
```

## 1.3 Copia manuale (alternativa)

Se preferisci copiare i file manualmente, consulta il campo `file` in `claude-marketplace/catalog.json` — è la fonte di verità del percorso su disco.

```
mkdir -p .claude/agents
cp claude-marketplace/stable/architecture/software-architect.md
.claude/agents/
```

Verifica anche il campo `"dependencies"` in `catalog.json` e copia le skill corrispondenti da `claude-marketplace/skills/<topic>/`.

## 1.4 Cos'è una skill?

Le skill sono knowledge provider atomici condivisi da più agenti. Non vengono invocate direttamente dall'utente — gli agenti le caricano internamente per accedere a standard e convenzioni (es. branding Accenture, pattern Java/Spring, regole REST). Lo setup script le gestisce automaticamente.

| Caratteristica | Agente                           | Skill                           |
|----------------|----------------------------------|---------------------------------|
| Invocazione    | Dall'utente o da un altro agente | Solo da un agente internamente  |
| Modello        | Sonnet o Opus                    | Haiku (sola lettura)            |
| Tool           | Read, Edit, Write, Bash, ...     | Solo <code>Read</code>          |
| Scopo          | Esegue task autonomamente        | Fornisce conoscenza strutturata |

## 2. Trovare la capability giusta

La tabella seguente riassume gli agenti di uso più frequente e quando invocarli.

| Capability                                  | Usa quando...                                                                             |
|---------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>software-architect</code>             | Progettare o revisionare un'architettura, scrivere ADR                                    |
| <code>functional-analyst</code>             | Estrarre requisiti, scrivere use case, mappare processi                                   |
| <code>developer-java</code>                 | Scrivere o revisionare codice Spring Boot                                                 |
| <code>developer-python</code>               | Scrivere o revisionare codice Python / FastAPI                                            |
| <code>developer-frontend</code>             | Sviluppo frontend multi-framework (Angular, React, Vue, Qwik, Vanilla)                    |
| <code>technical-analyst</code>              | Audit debito tecnico, security posture, code quality                                      |
| <code>code-reviewer</code>                  | Revisione di una PR o di un set di file modificati                                        |
| <code>test-writer</code>                    | Scrivere test per codice esistente (JUnit 5, pytest, Jest)                                |
| <code>debugger</code>                       | Diagnosi di un bug da stack trace + codice                                                |
| <code>api-designer</code>                   | Progettare o revisionare contract REST / OpenAPI                                          |
| <code>documentation-writer</code>           | Scrivere README, runbook, guide architetturali                                            |
| <code>presentation-creator</code>           | Creare presentazioni PowerPoint Accenture-branded                                         |
| <code>document-creator</code>               | Creare documenti PDF / DOCX Accenture-branded                                             |
| <code>orchestrator</code>                   | Task multi-dominio o ambigui — decompone e delega specialisti in parallelo                |
| <code>registry-auditor</code>               | Audit del registry contro le rubric Anthropic ufficiali                                   |
| <code>refactoring-supervisor</code>         | Workflow end-to-end di application replatforming (Fasi 0–4 completo)                      |
| <code>indexing-supervisor</code>            | Fase 0: indicizzazione codebase legacy in <code>.indexing-kb/</code>                      |
| <code>functional-analysis-supervisor</code> | Fase 1: analisi funzionale AS-IS + export PDF/PPTX                                        |
| <code>technical-analysis-supervisor</code>  | Fase 2: analisi tecnica AS-IS + export PDF/PPTX                                           |
| <code>baseline-testing-supervisor</code>    | Fase 3: suite di baseline regression AS-IS                                                |
| <code>deliberative-decision-engine</code>   | Decisioni complesse, ad alto rischio o irreversibili — pipeline di dibattito multi-agente |

**LEGACY:** `refactoring-tobe-supervisor` (ex Fase 4 big-bang) e `tobe-testing-supervisor` (ex Fase 5 equivalence) sono agenti LEGACY mantenuti solo per compatibilità

con i workflow v2. Il percorso canonico è `refactoring-supervisor` v3, che guida direttamente i passi 0–6 della Fase 4.



## 3. Verifica e configurazione

---

### 3.1 Verificare l'installazione

Apri Claude Code nel tuo progetto e digita `/agents` per vedere la lista degli agenti installati. Le capability aggiunte devono comparire nell'elenco.

In alternativa, avvia un task che attivi naturalmente la capability. Per esempio:

```
"Puoi revisionare l'architettura di questo servizio?"
```

Claude Code deve delegare automaticamente a `software-architect` in base alla sua descrizione.

### 3.2 Configurare i permessi (opzionale)

Per evitare la richiesta di conferma a ogni invocazione di un subagente, aggiungi regole esplicite

`Agent (name)` in `.claude/settings.json`:

```
{
  "permissions": {
    "allow": [
      "Agent (software-architect) ",
      "Agent (functional-analyst) ",
      "Agent (developer-java) "
    ]
  }
}
```

Lo setup script esegue questa operazione automaticamente. Se hai copiato i file manualmente, aggiungi le regole a mano.

## 4. Personalizzazione per progetto

Se una capability richiede contesto specifico del progetto (convenzioni di naming, framework proprietari, vocabolario di dominio), crea un override locale:

### Passo 1 — Copia la capability

```
cp .claude/agents/developer-java.md .claude/agents/developer-java-payments.md
```

### Passo 2 — Cambia il campo `name`

```
name: developer-java-payments
```

### Passo 3 — Aggiungi contesto alla fine del system prompt

```
## Project context (Payments Service)
- Package root: com.acme.payments
- Uses com.acme.commons.audit.AuditLogger
- All monetary amounts: BigDecimal, never double or float
```

**Mantieni gli override sottili.** Lo scopo è aggiungere contesto di progetto, non riscrivere la capability. Non includere logica operativa che appartiene alla capability condivisa.

### 4.1 Pinning a una versione specifica

Per default ottieni la versione corrente nel marketplace. Se il progetto deve restare su una versione specifica:

1. Verifica la versione in `claude-marketplace/catalog.json`
2. Fai checkout del tag git: `git checkout software-architect@1.0.0 -- claude-marketplace/stable/architecture/software-architect.md`
3. Copia quella versione nel tuo progetto
4. Documenta la versione pinned nel `CLAUDE.md` del progetto

## 5. Aggiornamenti

---

Quando vengono rilasciate nuove versioni delle capability nel registry, aggiorna il tuo progetto con:

```
# 1. Aggiorna il registry locale
git pull origin main

# 2. Re-installa le capability aggiornate
./claude-catalog/scripts/setup-capabilities.sh /percorso/tuo-progetto

# Oppure, per aggiornare l'installazione globale:
./claude-catalog/scripts/setup-capabilities.sh --global
```

Le versioni seguono SemVer: `PATCH` = fix, `MINOR` = nuovi comportamenti, `MAJOR` = breaking change. Una modifica al campo `name` è sempre breaking — aggiorna i riferimenti in `settings.json`.

**Attenzione:** Se hai override locali, verifica che siano ancora compatibili dopo un update MAJOR prima di re-installare. Gli override pinned non vengono sovrascritti dallo script.

PARTE 2

# Contribuire al Registry

---

Come creare una nuova capability, scrivere il system prompt, validare localmente, aprire una PR e pubblicare nel marketplace.

## 6. Struttura del repository

```
claude-registry/
  claude-catalog/          ← sorgente (sviluppo e review)
    agents/<topic>/        file .md agenti con YAML frontmatter
    skills/<topic>/        knowledge provider condivisi
    docs/<phase>/          ref docs caricati on-demand dai supervisor
    evals/<agent-name>/    suite di eval per agenti chiave
    templates/new-use-case/ scaffold per aggiungere un nuovo workflow
    scripts/
      setup-capabilities.sh installa le capability in un progetto
      new-capability.sh      scaffold nuova capability o skill
    CHANGELOG.md            ogni PR ha una voce [Unreleased]
    GOVERNANCE.md
    ANTI-PATTERNS.md       log di approcci deprecati

  claude-marketplace/      ← distribuzione (solo capability approvate)
    stable/<topic>/         capability production-ready
    beta/<topic>/           capability nuove o sperimentali
    skills/<topic>/         skill condivise
    catalog.json            manifest — campo "file" è la fonte di verità

  bmad/                   ← layer metodologia BMAD
    workflows.json         registry machine-readable dei workflow
    design/mapping.md      decisioni canoniche BMAD→Anthropic
    design/workflow-dag-draft.json grafo dipendenze agenti pipeline
    scripts/backfill-dag.py sincronizza campi bmad in catalog.json
```

### 6.1 La regola fondamentale: catalog vs marketplace

| Area                | Scopo                                                                   | Chi la modifica                 |
|---------------------|-------------------------------------------------------------------------|---------------------------------|
| claude-catalog/     | Sorgente di sviluppo — capability scritte, revisionate e versionate qui | Sviluppatori, PR                |
| claude-marketplace/ | Distribuzione — solo capability approvate, copiate dal catalog          | Solo publish script, mai a mano |

**Una capability non è "disponibile" finché non è pubblicata nel marketplace.** Modificare solo il catalog non è sufficiente.

# 7. Creare una nuova capability

## 7.1 Scaffold con il nuovo script

```
# Agente
./claude-catalog/scripts/new-capability.sh --type agent nome-agente

# Skill
./claude-catalog/scripts/new-capability.sh --type skill nome-skill
```

I template generati includono già:

- Frontmatter rubric-compliant ( `name` , `description` , `tools` , `model` , `color` )
- Prefisso descrizione nella forma corretta Anthropic
- Sezione `## When to invoke` con skeleton 2–4 bullet + `Do NOT use`
- Per le skill: clausola `Do not use...` e trigger phrases tra virgolette

## 7.2 Frontmatter obbligatorio

| Campo                    | Agente       | Skill                       | Note                                                                                              |
|--------------------------|--------------|-----------------------------|---------------------------------------------------------------------------------------------------|
| <code>name</code>        | Obbligatorio | Obbligatorio                | kebab-case, senza versione nel nome                                                               |
| <code>description</code> | Obbligatorio | Obbligatorio                | Vedi rubric sotto                                                                                 |
| <code>tools</code>       | Obbligatorio | <code>Read</code> fisso     | Skill: solo Read, niente altro senza giustificazione                                              |
| <code>model</code>       | Obbligatorio | <code>haiku</code><br>fisso | Agenti: <code>sonnet default</code> ; <code>opus</code> richiede <code>model_justification</code> |
| <code>color</code>       | Obbligatorio | Obbligatorio                | Solo: <code>red blue green yellow magenta cyan</code>                                             |

## 7.3 Rubric della descrizione (Anthropic)

Per gli agenti:

```
Use this agent when [condizioni].
Typical triggers include [scenario 1], [scenario 2], and [scenario 3].
See "When to invoke" in the agent body for worked scenarios.
```

Per le skill:

```
This skill should be used when [condizioni]. Typical triggers include
"[frase verbatim 1]", "[frase verbatim 2]", and "[frase verbatim 3]".
Do not use this skill for [scope-out clause].
```

## 8. Scrivere il system prompt

---

### 8.1 Struttura minima obbligatoria per gli agenti

1. `## Role` — un paragrafo che definisce l'identità dell'agente
2. `## When to invoke` — 2–4 bullet con scenari concreti + riga `Do NOT use this agent for:`
3. Sezioni operative: `## Method`, `## Outputs`, `## Constraints`, ecc.

### 8.2 Struttura per i supervisor (progressive disclosure)

I supervisor DEVONO contenere SOLO:

- `## Role` — un paragrafo
- `## When to invoke` — 3–4 bullet + `Do NOT use`
- `## Reference docs` — tabella con `Read when` per ogni doc di riferimento

Tutto il contenuto operativo (decision rules, escalation triggers, roster sub-agenti, output format) va nei reference doc sotto `claude-catalog/docs/<phase>/`.

### 8.3 Limiti di lunghezza del body

- **Agenti:** tetto di 10 000 caratteri (hard error per nuovi agenti)
- **Skill:** soft cap 3 000 parole, hard cap 5 000 parole (limite Anthropic)
- Agenti legacy con eccezioni documentate in `.github/scripts/legacy-body-baseline.json` — possono shrinkare ma non crescere

**Regola ratchet:** qualsiasi agente non nella baseline che supera 10 000 caratteri è un hard error in CI. Quando un file legacy scende sotto 10 000, rimuovilo subito dalla baseline per prevenire regressioni future.

### 8.4 Regola file-writing

Tutti gli agenti che producono file devono usare `Write` o `Edit`, mai `Bash` `heredoc/echo` `redirect/tee` per l'output di contenuto. Questo previene shell injection (specialmente con sintassi Mermaid su Windows/Git Bash).

## 9. Test e validazione locale

---

### 9.1 Due gate di validazione

```
# Gate 1 – valida claude-catalog/  
python3 .github/scripts/validate_catalog.py  
  
# Gate 2 – valida claude-marketplace/ (solo se gate 1 è verde)  
python3 .github/scripts/validate_marketplace.py
```

Gate 1 verifica:

- YAML frontmatter valido ( `name` , `description` , `tools` , `model` )
- System prompt con sezione `## Role`
- Skill senza tool proibiti ( `Edit` , `Write` , `Bash` , `Agent` )
- `CHANGELOG.md` ha una voce `[Unreleased]`
- Ogni agente/skill nel catalog ha una voce in `claude-marketplace/catalog.json`

Gate 2 verifica:

- `catalog.json` valido (semver, tier, status, campi obbligatori)
- Ogni file referenziato esiste su disco
- Il frontmatter `name` corrisponde alla voce catalog
- Convenzione path `{tier}/{<topic>}/{name}.md`
- Nessun file orfano in `stable/` , `beta/` , `skills/`

**Conseguenza pratica:** se apri una PR che aggiunge capability al catalog senza pubblicarle nel marketplace, gate 1 blocca la PR prima ancora che gate 2 parta.



# 10. PR, review e pubblicazione

## 10.1 Flusso di pubblicazione

```
claude-catalog/agents/<topic>/foo.md
└─ publish script → claude-marketplace/beta/<topic>/foo.md
                    claude-marketplace/catalog.json ← manifest

./claude-marketplace/scripts/publish.sh <name> <version> <tier> [--topic
<topic>]
```

Il publish script auto-risolve il `<topic>` da: (1) flag `--topic`, (2) voce `catalog.json` esistente, (3) path sorgente `catalog`. Rimuove anche eventuali copie stale della stessa capability nello stesso tier.

## 10.2 Regole obbligatorie prima di aprire la PR

- Voce `[Unreleased]` in `CHANGELOG.md` presente
- Voce in `catalog.json` aggiornata (versione, dipendenze, descrizione)
- Gate 1 e gate 2 verdi localmente
- Nessun file orfano nel marketplace

## 10.3 Promozione beta → stable

Sono supportati due criteri:

- **Criterio A (tempo):** uso in almeno 2 progetti + 30 giorni senza problemi critici
- **Criterio B (convergenza):** 2 o più specializzazioni indipendenti in project diversi (`.claude/agents/`) introducono la stessa modifica alla capability beta — la convergenza è un segnale di maturità più forte del tempo

## 10.4 Backfill DAG dopo aggiornamenti al workflow

```
python3 bmad/scripts/backfill-dag.py
```

Esegui dopo aver aggiornato il DAG draft in `bmad/design/workflow-dag-draft.json` per propagare i campi `bmad` in `catalog.json`.

# 11. Governance e versionamento

| Tipo di modifica                     | Cosa aggiornare                                                                                                                                               |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nuovo agente                         | <code>README.md</code> , <code>docs/quick-start.md</code> , <code>CHANGELOG.md</code><br><code>[Unreleased]</code> , <code>catalog.json</code>                |
| Nuova skill                          | <code>README.md</code> , <code>CHANGELOG.md</code> <code>[Unreleased]</code> , <code>catalog.json</code> (voce + dipendenze)                                  |
| Comportamento agente cambiato        | <code>CHANGELOG.md</code> <code>[Unreleased]</code> con bump versione, <code>catalog.json</code>                                                              |
| Nuovo workflow use case              | <code>bmad/workflows.json</code> , <code>catalog.json</code> (campi bmad),<br><code>evals/&lt;supervisor&gt;/triggers.json</code> , <code>CHANGELOG.md</code> |
| Body supervisor ridotto (extraction) | <code>docs/&lt;phase&gt;/supervisor-protocol.md</code> , rimuovi da <code>legacy-body-baseline.json</code>                                                    |
| Qualsiasi modifica significativa     | <code>guida-operativa.pdf</code> e <code>pitch-claude-registry.pptx</code> se il contenuto diventa stale                                                      |

## 11.1 SemVer e tag git

Il versionamento segue SemVer con tag git `name@MAJOR.MINOR.PATCH`. Una modifica al campo `name` è sempre breaking (MAJOR) perché Claude Code usa il nome per il routing di delega.

## 11.2 Anti-pattern log

Il file `claude-catalog/ANTI-PATTERNS.md` registra tutti gli approcci deprecati. Ogni PR di deprecazione deve lasciare una voce con: cosa è stato provato, perché non ha funzionato, cosa lo sostituisce, e le condizioni *Do not retry unless*. Il file sopravvive alla finestra di 90 giorni di removal.

PARTE 3

# Metodologia BMAD

---

Il layer BMAD adatta i pattern della metodologia BMAD al formato Anthropic single-file. Progressive disclosure, document-as-cache, workflow DAG, eval suite, e template per nuovi use case.

# 12. Cos'è il layer BMAD

Il registry adotta i *pattern di contenuto* della metodologia BMAD — non il suo formato a directory — mantenendo la compatibilità con il validator Anthropic che richiede file `.md` singoli e flat.

| Concetto BMAD                                                                      | Implementazione nel registry                                                                                             |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>SKILL.md</code> body                                                         | File <code>.md</code> singolo con <code>## Role</code> + <code>## When to invoke</code> + <code>## Reference docs</code> |
| <code>prompts/stage-NN.md</code>                                                   | <code>claude-catalog/docs/&lt;phase&gt;/supervisor-protocol.md</code> , <code>phase-plan.md</code> , ecc.                |
| <code>references/</code> directory                                                 | <code>claude-catalog/docs/&lt;phase&gt;/</code> companion docs                                                           |
| Progressive disclosure                                                             | Tabella <code>## Reference docs</code> con condizioni <code>Read when</code>                                             |
| Document-as-cache                                                                  | <code>_meta/pipeline-state.yaml</code> in ogni output directory di fase                                                  |
| <code>module-help.csv</code><br><code>preceded_by/</code> <code>followed_by</code> | Array <code>preceded_by[]</code> / <code>followed_by[]</code> nelle voci <code>catalog.json</code>                       |
| Module registry                                                                    | <code>bmad/workflows.json</code>                                                                                         |
| <code>triggers.json</code> + <code>evals.json</code>                               | <code>claude-catalog/evals/&lt;name&gt;/triggers.json</code> + <code>evals.json</code>                                   |
| Build Process (BP) scaffold                                                        | <code>claude-catalog/templates/new-use-case/</code>                                                                      |

I file di riferimento sono in `bmad/` :

- `bmad/workflows.json` — registry machine-readable di tutti i workflow disponibili
- `bmad/design/mapping.md` — decisioni canoniche di mapping BMAD→Anthropic
- `bmad/design/workflow-dag-draft.json` — grafo di dipendenze per tutti gli agenti pipeline
- `bmad/scripts/backfill-dag.py` — popola i campi `bmad` in `catalog.json` dal DAG

# 13. Progressive disclosure

Il problema: i body dei supervisor superavano 10 000 caratteri perché Decision rules, Escalation triggers, Sub-agent tables e Constraints erano inline. Il modello processava tutto al momento dell'attivazione indipendentemente da quale step stava eseguendo.

La soluzione BMAD: ogni supervisor mantiene nel suo `.md` body solo:

- `## Role` — un paragrafo massimo
- `## When to invoke` — 2–4 bullet + riga `Do NOT use`
- `## Reference docs` — tabella con file di riferimento e condizioni `Read when`

Tutto il contenuto operativo si sposta nei reference doc in `claude-catalog/docs/<phase>/`:

| Reference doc                            | Contiene                                                                           |
|------------------------------------------|------------------------------------------------------------------------------------|
| <code>supervisor-protocol.md</code>      | Decision rules, escalation triggers, constraints, manifest update instructions     |
| <code>phase-plan.md</code>               | Bootstrap dialog, dispatch per-wave, HITL checkpoint prompt, closing report schema |
| <code>sub-agents.md</code>               | Roster sub-agenti, wave assignments, output target, gate condizionali              |
| <code>dispatch-prompt-template.md</code> | Boilerplate del prompt per le chiamate Agent tool                                  |

## 13.1 Struttura directory dei reference doc

```
claude-catalog/docs/<phase>/
├── supervisor-protocol.md    # decision rules + escalation + constraints
├── phase-plan.md            # bootstrap + per-wave + HITL + closing
├── sub-agents.md            # roster + wave map + output targets
└── dispatch-prompt-template.md # Agent dispatch boilerplate
```

Valori di `<phase>`: `indexing`, `functional-analysis`, `technical-analysis`, `baseline-testing`, `refactoring-workflow`, `deliberation`.

**Risultato:** tutti i 18 supervisor e worker sono stati ridotti sotto 10 000 caratteri. Il validator è passato da 45 warning a 0 hard error. La `legacy-body-baseline.json` ora contiene solo il commento — nessuna eccezione attiva.

## 14. Document-as-cache (pipeline state)

Ogni fase scrive un file `_meta/pipeline-state.yaml` nella sua output directory. I supervisor lo leggono al bootstrap per rilevare lo stato di resume dopo un contesto compression event.

```
phase: "Phase 1 - Functional Analysis"
use_case: "application-replatforming"
run_id: "<uuid>"
started_at: "2026-05-30T10:00:00Z"
last_updated_at: "2026-05-30T10:45:00Z"
status: "in-progress" # pending | in-progress | complete | partial | failed
iteration: 1
waves:
  W1:
    status: complete
    completed_at: "2026-05-30T10:20:00Z"
    agents:
      - name: actor-feature-mapper
        status: complete
        output: docs/analysis/01-functional/raw/01-actors-features.md
  W2:
    status: in-progress
    agents: []
exports:
  pdf: null
  ptx: null
```

Protocollo per i supervisor:

1. Leggi `_meta/pipeline-state.yaml` al bootstrap (crea se assente)
2. Scrivi dopo ogni wave con gli status aggiornati
3. Usa il file per decidere cosa skippare al resume ( `status: complete` → skip) vs rieseguire
4. Sostituisce il pattern ad hoc di "detect existing outputs listando i file"

### Output directory per fase:

- Fase 0: `.indexing-kb/_meta/pipeline-state.yaml`
- Fase 1: `docs/analysis/01-functional/_meta/pipeline-state.yaml`
- Fase 2: `docs/analysis/02-technical/_meta/pipeline-state.yaml`
- Fase 3: `tests/baseline/_meta/pipeline-state.yaml`
- Fase 4: `.refactoring-kb/pipeline-state.yaml`

## 15. Workflow registry ( `bmad/workflows.json` )

---

`bmad/workflows.json` è il registry machine-readable di tutti i workflow disponibili. Ogni entry è un modulo auto-contenuto con il suo supervisor, le fasi, gli agenti condivisi e le trigger phrase. È la fonte di verità per la discovery dei workflow — non il body del supervisor.

```
{
  "version": "1.0.0",
  "workflows": [
    {
      "id": "application-replatforming",
      "name": "Application Replatforming",
      "entry_agent": "refactoring-supervisor",
      "status": "stable",
      "trigger_phrases": [
        "Start the application replatforming workflow",
        "Lancia il refactoring",
        "Run Phase", "Resume Phase"
      ],
      "phases": [ ... ],
      "shared_agents": [
        "developer-java", "developer-frontend", "test-writer",
        "debugger", "code-reviewer", "software-architect",
        "api-designer", "document-creator", "presentation-creator",
        "deliberative-decision-engine"
      ]
    }
  ]
}
```

Il campo `entry_agent` punta al supervisor del workflow. Per i supervisor di fase standalone (es. `indexing-supervisor`) l'entry agent è il supervisor stesso. Il campo `shared_agents` lista gli agenti usati trasversalmente da più fasi.

## 16. DAG in catalog.json

---

Ogni agente pipeline in `catalog.json` porta un blocco `bmad` con i metadati del grafo di dipendenza:

```
"bmad": {
  "workflow": "application-replatforming",
  "role": "worker",
  "phase": "phase-1",
  "wave": "W2",
  "preceded_by": ["actor-feature-mapper", "ui-surface-analyst"],
  "followed_by": ["functional-analysis-challenger"]
}
```

Campi:

- `preceded_by`: agenti i cui output questo agente consuma (devono essere completi prima)
- `followed_by`: agenti che consumano l'output di questo agente
- `workflow`: workflow di appartenenza (vuoto per agenti standalone)
- `phase`: identificatore della fase nel workflow
- `wave`: identificatore della wave nella fase

**Manutenzione:** dopo aver aggiornato `bmad/design/workflow-dag-draft.json`, esegui `python3 bmad/scripts/backfill-dag.py` per propagare i campi `bmad` nel `catalog.json`. Gli agenti standalone portano un blocco `bmad` vuoto.



# 17. Eval suite

---

Ogni supervisor e agente standalone chiave ha:

- `claude-catalog/evals/<name>/triggers.json` — 2+ test di trigger positivi + 2+ negativi
- `claude-catalog/evals/<name>/evals.json` — aspettative end-to-end sugli artefatti prodotti (per i supervisor)

## 17.1 Schema triggers.json

```
[
  {
    "query": "Index this codebase and build the knowledge base",
    "should_trigger": true,
    "description": "Direct Phase 0 invocation"
  },
  {
    "query": "What does this function do?",
    "should_trigger": false,
    "description": "Generic question – should NOT trigger indexing-supervisor"
  }
]
```

## 17.2 Schema evals.json

```
[
  {
    "id": "eval-001",
    "prompt": "Index the codebase at /path/to/repo",
    "expectations": [
      "Dispatches codebase-mapper in Wave 1",
      "Produces .indexing-kb/bronze/stack.json",
      "Runs indexing-auditor before HITL",
      "Asks user before overwriting existing index"
    ],
    "timeout": 300
  }
]
```

Ordine di priorità per la scrittura delle eval:

1. Tutti i 6 supervisor di fase (trigger + artifact eval)
2. Top-10 agenti standalone per frequenza d'uso
3. Tutti gli agenti challenger/auditor (solo artifact eval — l'attivazione avviene via supervisor)

## 18. Aggiungere un nuovo use case

---

Per aggiungere un workflow use case oltre all'"application replatforming", usa lo scaffold in `claude-catalog/templates/new-use-case/` e segui la guida in 8 passi del README incluso:

### Step 1

Definisci l'ID del use case, il nome, le trigger phrase

### Step 2

Copia `supervisor-template.md` in `claude-catalog/agents/orchestration/<use-case>-supervisor.md`

### Step 3

Identifica i worker — quali sono condivisi (da `shared_agents`) e quali sono nuovi

### Step 4

Per ogni nuovo worker: copia `worker-template.md` e compila il body

### Step 5

Crea `claude-catalog/docs/<use-case>/` con i reference doc

### Step 6

Aggiungi una voce workflow a `bmad/workflows.json`

### Step 7

Aggiungi voci a `catalog.json` per supervisor e tutti i nuovi worker

### Step 8

Pubblica nel marketplace con `./claude-marketplace/scripts/publish.sh` e aggiorna il CHANGELOG

**Regola chiave:** aggiungi sempre prima la voce in `bmad/workflows.json` — prima di scrivere qualsiasi agente. Questo è il punto di ancoraggio per tutta la struttura successiva.

# 19. Il workflow Application Replatforming

L'unico workflow registrato è "Application Replatforming" (ID: `application-replatforming`, entry agent: `refactoring-supervisor` ). Copre la migrazione end-to-end AS-IS → TO-BE di qualsiasi codebase legacy.

| Fase                               | Supervisor                                  | Output directory                                                                | Standalone?                            |
|------------------------------------|---------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------|
| Fase 0 — Codebase Indexing         | <code>indexing-supervisor</code>            | <code>.indexing-kb/</code>                                                      | Sì                                     |
| Fase 1 — Functional Analysis       | <code>functional-analysis-supervisor</code> | <code>docs/analysis/01-functional/</code>                                       | Sì                                     |
| Fase 2 — Technical Analysis        | <code>technical-analysis-supervisor</code>  | <code>docs/analysis/02-technical/</code>                                        | Sì                                     |
| Fase 3 — Baseline Testing          | <code>baseline-testing-supervisor</code>    | <code>tests/baseline/</code>                                                    | Sì (richiede Fasi 0+1)                 |
| Fase 4 — Application Replatforming | <code>refactoring-supervisor</code>         | <code>backend/</code> , <code>frontend/</code> , <code>docs/refactoring/</code> | No (richiede tutte le fasi precedenti) |

## 19.1 Struttura Fase 4 (Steps 0–6)

La Fase 4 è guidata direttamente da `refactoring-supervisor` che usa agenti fine-grained per osservare ogni gate. Non c'è più un sub-supervisor separato per la replatforming:

| Step | Nome                            | Descrizione                                                                                                                                                                                                  |
|------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Bootstrap hard gate             | Scaffold progetto + build + startup: devono riuscire prima di procedere                                                                                                                                      |
| 1    | Minimal runnable skeleton       | Controller vuoti, pagine blank, security semplificata — app sempre buildable                                                                                                                                 |
| 2    | Incremental feature loop        | Per ogni UC: implementa → test → build → run → valida vs oracle Fase 3; avanza solo se tutto passa                                                                                                           |
| 3    | Mandatory validation loop       | Sub-loop automatico su ogni fallimento — root cause + fix minimo + re-valida; blocca il progresso                                                                                                            |
| 4    | Progressive system construction | Vertical slice, no astrazione prematura, app sempre buildable/testabile                                                                                                                                      |
| 5    | Hardening                       | Spring Security 6, config env, RFC 7807, JSON logging, CSP — re-validato dopo ogni modifica                                                                                                                  |
| 5.5  | Test data seeding               | <b>Nuovo (v3):</b> carica dataset cross-modulo coerente nell'app TO-BE prima del gate visivo. Usato da <code>test-data-seeder</code> . Non skippabile — senza dati, il UI smoke gate non può giudicare l'app |
| 6    | Final validation                | Suite test completa + business-flow validation vs oracle Fase 3 + TODO sweep + deliverable <code>01-replatforming-report.md</code> con sign-off PO                                                           |

**Phase 5 rimossa:** non esiste più una "Fase 5 TO-BE Testing & Equivalence Verification" separata. L'equivalence testing è assorbito nel Step 6 della Fase 4. Gli agenti `refactoring-tobe-supervisor` e `tobe-testing-supervisor` sono LEGACY (v2 only) — mantenuti per compatibilità, non nel percorso canonico.

## 19.2 HITL (Human-in-the-Loop)

Il workflow ha checkpoint HITL espliciti:

- Tra ogni fase (prompt esplicito skip / re-run / revise / iterate)
- Dopo ogni step della Fase 4 (prompt next / pause / revise / stop)
- Al termine della Fase 4 Step 6: blocca finché ci sono failure critici/alti o TODO in sospeso
- Fasi 1–3: loop iterate approvato con `approve / iterate / stop` prima di avanzare

## 19.3 Deliberazione (multi-agent debate)

Il `deliberative-decision-engine` è disponibile come agente trasversale per decisioni complesse, ad alto rischio o irreversibili. Può essere attivato:

- Con trigger espliciti IT/EN: "decidi con dibattito", "debate mode", "multi-agent debate", "decision review", ...
- Programmaticamente via `decisionMode: deliberative` nel JSON di dispatch
- Per auto-escalation del supervisor su decisioni di Phase 4 compliance-sensitive

Produce un audit trail completo in `.deliberation-kb/<trace-id>/`.

APPENDICE

# Tabella Capability di Riferimento

---

Elenco completo degli 87 agenti e 42 skill presenti nel registry al 31 maggio 2026.

## A. Agenti (87)

### Orchestrazione e supervisione

| Nome                                      | Tier        | Descrizione sintetica                                                                                                                 |
|-------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>orchestrator</code>                 | <b>beta</b> | Meta-orchestratore opus: decompone task multi-dominio, dispatcha specialisti in parallelo, sintetizza risultati                       |
| <code>refactoring-supervisor</code>       | <b>beta</b> | Workflow supervisor end-to-end replatforming (Fasi 0–4, v3.0.0): guida direttamente i 7 step della Fase 4 senza sub-supervisor legacy |
| <code>deliberative-decision-engine</code> | <b>beta</b> | Pipeline di dibattito multi-agente 7 step per decisioni complesse/irreversibili                                                       |

### Agenti standalone (quality, doc, API, architettura)

| Nome                              | Tier          | Descrizione sintetica                                              |
|-----------------------------------|---------------|--------------------------------------------------------------------|
| <code>software-architect</code>   | <b>stable</b> | Analisi architetturale, ADR, trade-off evaluation                  |
| <code>functional-analyst</code>   | <b>stable</b> | Requisiti, use case, processi di business                          |
| <code>technical-analyst</code>    | <b>beta</b>   | Debito tecnico, security posture, dipendenze vulnerabili           |
| <code>code-reviewer</code>        | <b>beta</b>   | Revisione strutturata di PR o file modificati                      |
| <code>test-writer</code>          | <b>beta</b>   | JUnit 5, Mockito, Testcontainers, pytest                           |
| <code>debugger</code>             | <b>beta</b>   | Diagnosi bug da stack trace, log e codice                          |
| <code>api-designer</code>         | <b>beta</b>   | Progettazione REST API e spec OpenAPI                              |
| <code>documentation-writer</code> | <b>beta</b>   | README, runbook, guide architetturali                              |
| <code>wiki-writer</code>          | <b>beta</b>   | GitHub wiki strutturate con framework Diataxis                     |
| <code>presentation-creator</code> | <b>beta</b>   | Slide PPTX Accenture-branded da documenti di progetto              |
| <code>document-creator</code>     | <b>beta</b>   | Documenti PDF/DOCX Accenture-branded da documenti di progetto      |
| <code>registry-auditor</code>     | <b>beta</b>   | Audit del registry contro le rubric Anthropic ufficiali; read-only |

### Developer agents (9)

| Nome                            | Tier          | Stack                                                       |
|---------------------------------|---------------|-------------------------------------------------------------|
| <code>developer-java</code>     | <b>stable</b> | Java / Spring Boot 3 enterprise development                 |
| <code>developer-python</code>   | <b>beta</b>   | Python / FastAPI                                            |
| <code>developer-frontend</code> | <b>beta</b>   | Angular, React/Next.js/TanStack, Vue 3, Qwik, Vanilla JS/TS |

| Nome             | Tier | Stack                                                  |
|------------------|------|--------------------------------------------------------|
| developer-go     | beta | Go — net/http, chi, gin, cobra                         |
| developer-rust   | beta | Rust stable — axum, actix-web, tokio, tracing          |
| developer-kotlin | beta | Kotlin JVM — Spring Boot 3 con idiomi Kotlin, Ktor     |
| developer-csharp | beta | C# / .NET 8+ — ASP.NET Core, EF Core                   |
| developer-ruby   | beta | Ruby on Rails 7+ — service/form/query objects, RSpec   |
| developer-php    | beta | PHP 8.2+ — Laravel 10/11, Symfony 6/7, PHPStan level 8 |

## Deliberation personas (6)

| Nome                            | Ruolo nel dibattito                                                  |
|---------------------------------|----------------------------------------------------------------------|
| debate-proposer                 | Primary Architect / Proposer — produce la lead recommendation        |
| debate-critic                   | Skeptical Critic — assumption audit, failure modes, edge cases       |
| debate-replatforming-specialist | Migration Specialist — sequencing, cutover, rollback, data migration |
| debate-risk-reviewer            | Security / Compliance / Risk Reviewer — GDPR/HIPAA/PCI/SOX           |
| debate-operations-reviewer      | Operations / Reliability Reviewer — SLO, capacity, RTO/RPO           |
| debate-judge                    | Neutral judge — SUMMARISER (Step 3) e ARBITRATOR (Step 6)            |

## Fase 0 — Indexing pipeline (8 agenti)

| Nome                   | Wave       | Funzione                                                          |
|------------------------|------------|-------------------------------------------------------------------|
| indexing-supervisor    | Supervisor | Entrypoint unico, orchestra 7 worker in 4 wave + indexing-auditor |
| codebase-mapper        | W1         | Inventario strutturale — tree, LOC, endpoint, bronze/KB           |
| dependency-analyzer    | W1         | Dipendenze esterne + import graph + cicli                         |
| streamlit-analyzer     | W1 (cond.) | Analisi Streamlit — pagine, session_state, widget, caching        |
| module-documenter      | W2 fan-out | Documentazione API per-package (parallelizzabile)                 |
| data-flow-analyst      | W3         | DB, API esterni, file I/O, env vars, configurazione               |
| business-logic-analyst | W3         | Concetti di dominio, validation rule, state machine               |
| synthesizer            | W4         | Consolidamento finale — overview, bounded context, hotspot        |
| indexing-auditor       | W4a        | Audit coverage gap, orphan file, AS-IS purity violations          |

## Fase 1 — Functional Analysis (6 agenti)

| Nome                            | Wave          | Funzione                                                          |
|---------------------------------|---------------|-------------------------------------------------------------------|
| functional-analysis-supervisor  | Supervisor    | v3+: iterative HITL loop, export wave PDF+PPTX, feature narrative |
| actor-feature-mapper            | W1            | Attori, ruoli, personas, feature map, matrice Actor×Feature       |
| ui-surface-analyst              | W1            | Schermate, navigation graph, component tree (Streamlit-aware)     |
| io-catalog-analyst              | W1            | Input, output, transformation matrix (prospettiva funzionale)     |
| user-flow-analyst               | W2            | Use case, user flow, sequence diagram Mermaid                     |
| implicit-logic-analyst          | W2            | Validation nascosta, state machine, magic number, silent fallback |
| functional-analysis-challenger  | W3 opt-in     | Revisione avversariale — gap, contraddizioni, violazioni AS-IS    |
| functional-traceability-auditor | W3b always-ON | Audit grounding evidenze — confirmed UC con evidence_ids          |

## Fase 2 — Technical Analysis (11 agenti)

| Nome                          | Wave          | Funzione                                                                    |
|-------------------------------|---------------|-----------------------------------------------------------------------------|
| technical-analysis-supervisor | Supervisor    | v2+: iterative HITL, export wave, adaptive dispatch mode                    |
| code-quality-analyst          | W1            | Codebase map, duplicazione, complexity hotspot, monolith smell              |
| state-runtime-analyst         | W1            | Session state, globals, side effect, state-flow diagram                     |
| dependency-security-analyst   | W1            | Inventario dipendenze, CVE, deprecation, licenze, SBOM-lite                 |
| data-access-analyst           | W1            | Data flow, DB/file/cache/serialization access pattern                       |
| integration-analyst           | W1            | Integrazioni esterne, auth/timeout/retry, integration map                   |
| performance-analyst           | W1            | N+1, hot loop, blocking I/O, caching gap (solo analisi statica)             |
| resilience-analyst            | W1            | Error handling, logging, silent failure, fallback chain                     |
| security-analyst              | W1            | OWASP Top 10, input validation, secret in codice, STRIDE                    |
| risk-synthesizer              | W2            | Risk register unificato MD/JSON/CSV, severity matrix                        |
| technical-analysis-challenger | W3 always-ON  | 8 check avversariali — gap, contraddizioni, violazioni AS-IS                |
| technical-evidence-auditor    | W3b always-ON | Audit grounding evidenze — finding con evidence_ids, high/critical verified |



## Fase 3 — Baseline Testing (8 agenti)

| Nome                        | Wave         | Funzione                                                            |
|-----------------------------|--------------|---------------------------------------------------------------------|
| baseline-testing-supervisor | Supervisor   | v2+: iterative HITL loop, oracle-guarded, adaptive execution policy |
| fixture-builder             | W0           | conftest.py con determinismo seed/time/network + fixture            |
| usecase-test-writer         | W1 fan-out   | Un modulo pytest per UC (happy + alternative + edge)                |
| integration-test-writer     | W1           | Test DB / file system / API esterne (mockate)                       |
| benchmark-writer            | W1           | pytest-benchmark + memory profiling — oracle performance AS-IS      |
| service-collection-builder  | W1 cond.     | Postman 2.1 collection per servizi esposti dall'app AS-IS           |
| baseline-runner             | W2           | Esegue la suite, cattura snapshot/benchmark JSON/coverage           |
| baseline-challenger         | W3 always-ON | 7 check avversariali su coverage, AS-IS purity, oracle integrity    |

## Fase 4 — TO-BE Refactoring (sub-agenti LEGACY v2)

Questi agenti sono mantenuti per compatibilità con workflow v2. Il percorso canonico v3 usa `refactoring-supervisor` con agenti fine-grained diretti.

| Nome                        | Funzione (v2)                                                              |
|-----------------------------|----------------------------------------------------------------------------|
| refactoring-tobe-supervisor | LEGACY: Supervisor big-bang Fase 4 v2 (6 wave, checkpoint HITL)            |
| decomposition-architect     | Decomposizione bounded context DDD, ADR-001+ADR-002                        |
| api-contract-designer       | OpenAPI 3.1 contract, Postman TO-BE collection, ADR-003                    |
| backend-scaffolder          | Scaffold Spring Boot 3 Maven, controller da OpenAPI, DTO, service          |
| data-mapper                 | JPA entity, Liquibase YAML changelog, Spring Data JPA repository           |
| logic-translator            | Traduce un UC AS-IS Python → Java/Spring (fan-out per UC)                  |
| frontend-scaffolder         | Angular 17+ workspace, standalone components, OpenAPI client tipato        |
| hardening-architect         | Observability (JSON logging, Micrometer, OpenTelemetry), security baseline |
| migration-roadmap-builder   | Roadmap strangler-fig, milestones per BC, rollback plan                    |
| phase4-challenger           | Matrice AS-IS↔TO-BE traceability + 10 check avversariali                   |

## Fase 4 Step 5.5 — Test Data Seeding (nuovo)

| Nome             | Funzione                                                                                                                                        |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| test-data-seeder | Carica dataset cross-modulo coerente nel TO-BE app prima del UI smoke gate. Auto-rileva il migration tool (13 tool supportati). Non skippabile. |

## TO-BE Testing (LEGACY v2 — 8 agenti)

La Fase 5 equivalence testing è assorbita in Fase 4 Step 6 in v3. Questi agenti sono mantenuti solo per workflow v2.

| Nome                    | Funzione (v2)                                                             |
|-------------------------|---------------------------------------------------------------------------|
| tobe-testing-supervisor | LEGACY: Supervisor Fase 5 — final go-live gate v2                         |
| equivalence-test-writer | Harness pytest TO-BE vs snapshot Phase 3 (HTTP o Playwright)              |
| backend-test-writer     | JUnit 5 + Mockito + Testcontainers + Spring Cloud Contract                |
| frontend-test-writer    | Jest + Angular Testing Library + Playwright E2E                           |
| security-test-writer    | OWASP Top 10 A01-A10, auth/authz, headers/CORS                            |
| performance-comparator  | Gatling/k6 — delta p95/p99 vs Fase 3 baseline                             |
| tobe-test-runner        | Esegue suite, cattura coverage, contract verifier, Playwright traces      |
| equivalence-synthesizer | Produce <code>01-equivalence-report.md</code> (DELIVERABLE — PO sign-off) |
| tobe-testing-challenger | 8 check avversariali su UC coverage, drift, AS-IS-only leak               |

## B. Skill (42)

Le skill sono knowledge provider atomici (model: haiku, tools: Read) condivisi da più agenti. Non si invocano direttamente — vengono caricate internamente dagli agenti.

| Nome                       | Topic            | Usata da                                                     |
|----------------------------|------------------|--------------------------------------------------------------|
| java-spring-standards      | backend          | developer-java, code-reviewer, test-writer                   |
| spring-expert              | backend          | developer-java                                               |
| spring-architecture        | backend          | developer-java, software-architect                           |
| spring-data-jpa            | backend          | developer-java, data-mapper                                  |
| java-expert                | backend          | developer-java                                               |
| postgresql-expert          | database         | developer-java, data-mapper                                  |
| testing-standards          | testing          | developer-java, test-writer, code-reviewer, developer-python |
| rest-api-standards         | api              | developer-java, api-designer, code-reviewer                  |
| accenture-branding         | branding         | presentation-creator, document-creator                       |
| functional-reconstruction  | analysis         | developer-frontend, functional-analyst                       |
| tech-analyst               | analysis         | technical-analyst, software-architect                        |
| refactoring-expert         | refactoring      | developer-java, code-reviewer                                |
| doc-expert                 | documentation    | documentation-writer                                         |
| backend-documentation      | documentation    | documentation-writer                                         |
| frontend-documentation     | documentation    | documentation-writer, wiki-writer                            |
| documentation-orchestrator | orchestrators    | documentation-writer                                         |
| backend-orchestrator       | orchestrators    | orchestrator                                                 |
| frontend-orchestrator      | orchestrators    | orchestrator                                                 |
| angular-expert             | frontend/angular | developer-frontend                                           |
| ngrx-expert                | frontend/angular | developer-frontend                                           |
| rxjs-expert                | frontend/angular | developer-frontend                                           |
| react-expert               | frontend/react   | developer-frontend                                           |
| tanstack                   | frontend/react   | developer-frontend                                           |
| tanstack-query             | frontend/react   | developer-frontend                                           |
| tanstack-start             | frontend/react   | developer-frontend                                           |
| nextjs                     | frontend/react   | developer-frontend                                           |

| Nome                                                                    | Topic            | Usata da                                            |
|-------------------------------------------------------------------------|------------------|-----------------------------------------------------|
| vue-expert                                                              | frontend/vue     | developer-frontend                                  |
| qwik-expert                                                             | frontend/qwik    | developer-frontend                                  |
| vanilla-expert                                                          | frontend/vanilla | developer-frontend                                  |
| css-expert                                                              | frontend/vanilla | developer-frontend                                  |
| design-expert                                                           | frontend         | developer-frontend                                  |
| unicredit-design-system                                                 | frontend         | developer-frontend (auto-load per client UniCredit) |
| python-expert                                                           | python           | developer-python                                    |
| dependency-resolver                                                     | utils            | orchestrator                                        |
| uml-diagram-generator                                                   | documentation    | documentation-writer                                |
| browser-automation                                                      | testing          | developer-frontend, test-writer                     |
| test-data-seeding-standards                                             | testing          | test-data-seeder                                    |
| streamlit-expert                                                        | python           | developer-python, indexing sub-agenti               |
| + 3 skill utils/orchestrators (caveman, caveman-review, caveman-commit) |                  |                                                     |

## C. Comandi di riferimento rapido

| Operazione                   | Comando                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------|
| Installazione in un progetto | <code>./claude-catalog/scripts/setup-capabilities.sh /percorso/progetto</code>                 |
| Installazione globale        | <code>./claude-catalog/scripts/setup-capabilities.sh --global</code>                           |
| Scaffold nuovo agente        | <code>./claude-catalog/scripts/new-capability.sh --type agent &lt;nome&gt;</code>              |
| Scaffold nuova skill         | <code>./claude-catalog/scripts/new-capability.sh --type skill &lt;nome&gt;</code>              |
| Pubblica nel marketplace     | <code>./claude-marketplace/scripts/publish.sh &lt;name&gt; &lt;version&gt; &lt;tier&gt;</code> |
| Valida catalog (gate 1)      | <code>python3 .github/scripts/validate_catalog.py</code>                                       |
| Valida marketplace (gate 2)  | <code>python3 .github/scripts/validate_marketplace.py</code>                                   |
| Backfill DAG in catalog.json | <code>python3 bmad/scripts/backfill-dag.py</code>                                              |

# D. Convenzioni di naming e colori

## D.1 Nomi file

- kebab-case senza versione: `developer-java.md` , non `developer-java-v2.md`
- Agenti raggruppati per topic in `agents/<topic>/`
- Skill raggruppate in `skills/<topic>[/<subtopic>]/`

## D.2 Colori frontmatter validi (spec Anthropic)

| Valore               | Note                                                 |
|----------------------|------------------------------------------------------|
| <code>red</code>     | Usato da: refactoring-tobe agents                    |
| <code>blue</code>    | Usato da: agenti di analisi API                      |
| <code>green</code>   | Usato da: baseline-testing agents                    |
| <code>yellow</code>  | Usato da: technical-analysis agents                  |
| <code>magenta</code> | Usato da: indexing agents                            |
| <code>cyan</code>    | Usato da: functional-analysis agents, quality agents |

I valori `purple` , `orange` , `pink` non sono validi nella spec Anthropic e vengono rifiutati dal validator.

## D.3 Modelli

| Modello             | Uso                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------|
| <code>haiku</code>  | Tutte le skill (obbligatorio)                                                                       |
| <code>sonnet</code> | Agenti standard (default consigliato)                                                               |
| <code>opus</code>   | Solo meta-orchestratore (auto-allowed) o agenti con <code>model_justification</code> $\geq$ 40 char |

## D.4 Topic folder del marketplace

**Agenti:** `analysis` , `api` , `architecture` , `baseline-testing` , `developers` , `documentation` , `functional-analysis` , `indexing` , `orchestration` , `quality` , `refactoring-tobe` , `technical-analysis` , `tobe-testing`

**Skill:** `analysis` , `api` , `backend` , `branding` , `database` , `documentation` , `frontend` (con subdir per framework) , `orchestrators` , `python` , `refactoring` , `testing` , `utils`

Crea un nuovo topic folder quando una terza capability dello stesso tipo appare. Fino ad allora usa la folder più vicina esistente.

